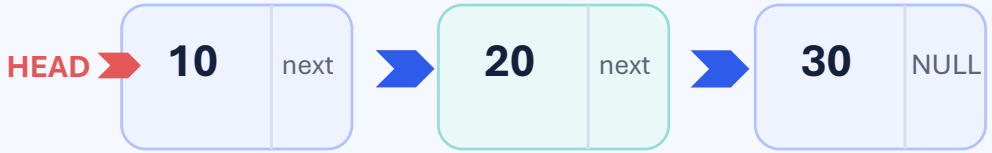
Single Linked List

Konsep, operasi dasar, analisis kompleksitas, dan implementasi C++

Struktur Dinamis | Pointer | Implementasi C++

Muhammad Zainudin Al Amin
Zainudin@unimus.ac.id
Mata Kuliah truktur Data

Ilustrasi node



Deck ini dirancang sebagai materi presentasi yang lebih lengkap dari ringkasan sebelumnya: mulai dari definisi, struktur node, operasi dasar, hingga program C++ dan aplikasi praktis.

Peta Materi

Alur pembahasan dari konsep dasar hingga implementasi

01 **Definisi & karakteristik**
Mengapa linked list dipakai dan bagaimana node saling terhubung.

02 **Struktur node**
Peran head, data, dan pointer next pada singly linked list.

03 **Jenis linked list**
Non-circular vs circular serta kaitannya dengan doubly linked list.

04 **Operasi dasar**
Traversal, insert, delete, search, update, dan clear.

05 **Implementasi C++**
Kode program lengkap berbasis menu.

06 **Analisis & aplikasi**
Kompleksitas waktu, kelebihan, keterbatasan, dan contoh penggunaan.

Apa itu Single Linked List?

Struktur data linear dinamis yang disusun oleh node-node yang saling terhubung

Definisi inti

Single linked list terdiri dari node yang masing-masing menyimpan data dan sebuah pointer next menuju node berikutnya. Karena tiap node hanya memiliki satu pointer, traversal hanya dapat dilakukan dari depan ke belakang.

Karakteristik

- ukuran dinamis saat runtime
- memori tidak harus berurutan
- sisip/hapus node relatif fleksibel
- akses elemen harus melalui traversal

Linked list vs array

Aspek	Array	Single Linked List
Lokasi memori	Kontigu	Tidak harus kontigu
Ukuran	Cenderung tetap	Dinamis
Akses elemen	Cepat via indeks	Traversal O(n)
Insert/Delete tengah	Perlu geser elemen	Lebih fleksibel jika posisi diketahui

SINGLE LINKED LIST

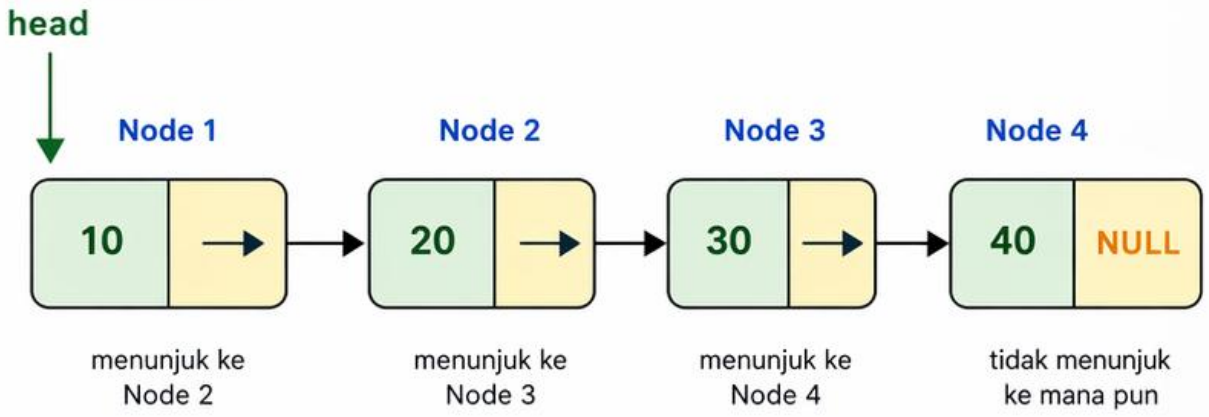
Struktur Data Linear Dinamis

Sekumpulan node yang saling terhubung menggunakan satu pointer penghubung (*next*).

SETIAP NODE MENYIMPAN:

- **data** : nilai yang disimpan dalam node
- **next** : pointer yang menunjuk ke node berikutnya

ILUSTRASI:



KETERANGAN:

data
Nilai yang disimpan

next
Pointer ke node berikutnya

NULL Menandakan akhir list (tidak ada node berikutnya)

KONSEP UTAMA:

Pada **single linked list**, setiap node hanya memiliki **satu pointer**, yaitu **next**. Karena itu, pergerakan traversal hanya dapat dilakukan **satu arah**, yaitu **dari depan ke belakang**, mulai dari **head** hingga mencapai **NULL**.

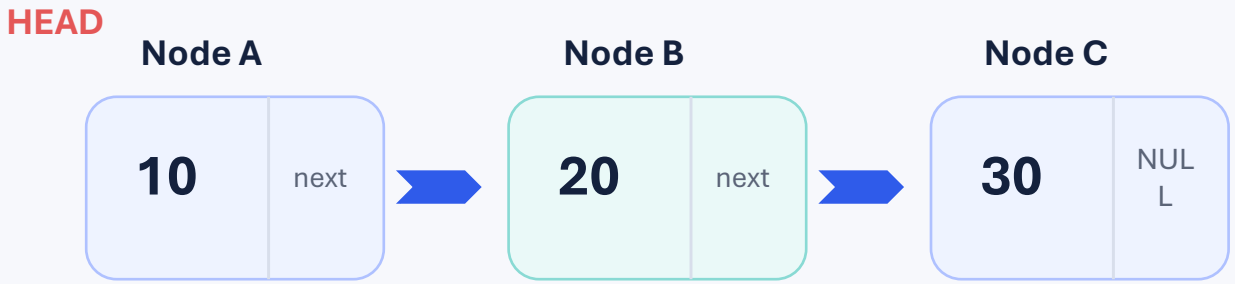
ANALOGI:

Seperti antrean orang yang hanya bisa bergerak maju. Setiap orang (node) hanya tahu siapa yang ada di depannya, tapi tidak tahu siapa yang di belakangnya.

Struktur Node dan Peran Pointer

Bagaimana head, data, dan next bekerja di dalam singly linked list

Diagram alur keterkaitan node



Intinya: setiap node mengetahui siapa penerusnya, tetapi tidak mengetahui node sebelumnya. Inilah alasan singly linked list hanya mendukung traversal satu arah.

Makna tiap bagian

Head	pointer awal yang selalu menunjuk node pertama
Data	nilai/informasi yang disimpan oleh node
Next	alamat menuju node berikutnya
NULL	penanda node terakhir pada list non-circular

Varian dan Aplikasi Linked List

Singly linked list adalah fondasi sebelum mempelajari circular dan doubly linked list

Non-circular

Node terakhir menunjuk NULL. Ini adalah bentuk dasar yang paling umum dipakai untuk pengenalan linked list dan banyak latihan praktikum.

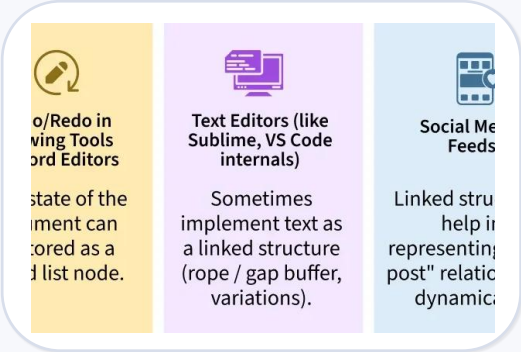
```
HEAD → [10|next] → [20|next]
→ [30|NULL]
```

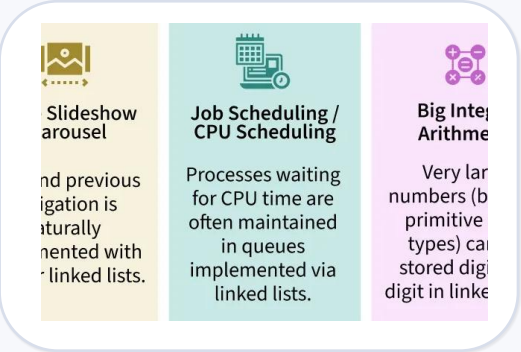
Circular singly linked list

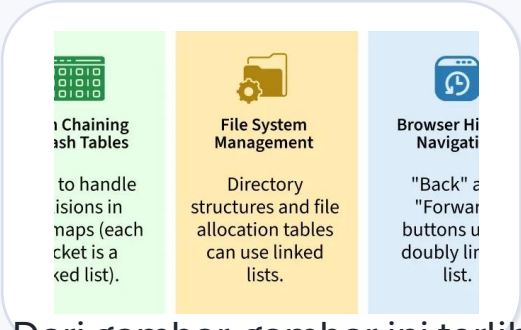
Node terakhir menunjuk kembali ke head, sehingga traversal bersifat melingkar. Cocok untuk proses berulang seperti rotasi atau antrean melingkar.

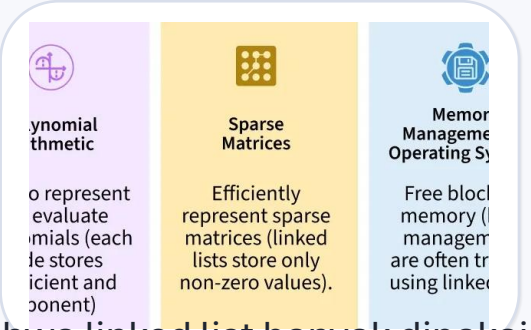
```
HEAD → [10] → [20] → [30] ↺
head
```

Contoh aplikasi praktis









Dari gambar-gambar ini terlihat bahwa linked list banyak dipakai pada konteks antrian proses, hash chaining, sparse matrix, polynomial arithmetic, manajemen memori, dan circular navigation. Untuk browser history atau undo/redo, varian doubly linked list biasanya lebih sesuai.

SINGLE LINKED LIST: NON-CIRCULAR vs CIRCULAR

Memahami Perbedaan Pointer pada Single Linked List

A SINGLE LINKED LIST NON-CIRCULAR

Pada bentuk ini, node terakhir menunjukkan ke `NULL`.

Ilustrasi:

Representasi: `HEAD -> [10|•] -> [20|•] -> [30|•] -> NULL`

KESIMPULAN:
Bentuk ini adalah bentuk dasar singly linked list. Modul juga menunjukkan bahwa node terakhir pada single linked list non-circular bernilai `NULL`.

B SINGLE LINKED LIST CIRCULAR

Pada bentuk circular, node terakhir tidak menunjuk `NULL`, tetapi kembali menunjuk ke **head**.

Ilustrasi:

Representasi:

```
HEAD -> [10|•] -> [20|•] -> [30|•]
 ^                                |
 |________________________________|
```

KESIMPULAN:
Modul menjelaskan bahwa pada circular singly linked list, pointer `next` dari node terakhir diarahkan kembali ke **node kepala**.

INTI KONSEP: Perbedaan utama terletak pada nilai pointer `next` di node terakhir: `NULL` untuk non-circular, dan **head** untuk circular.

Operasi Dasar Single Linked List

Inti manipulasi pointer yang wajib dikuasai di praktikum Struktur Data

01 Traversal
menelusuri node dari head sampai NULL

02 Insert depan
menjadikan node baru sebagai head

03 Insert belakang
menambahkan node di akhir list

04 Insert setelah key
menyisipkan node setelah data tertentu

05 Delete depan
memindahkan head ke node berikutnya

06 Delete belakang
mencari node sebelum terakhir lalu memutus link

07 Search
mencari data melalui traversal

08 Update
mengubah isi node yang ditemukan

09 Clear
menghapus semua node dan membebaskan memori

Kesalahan paling umum pada operasi linked list adalah salah mengatur pointer next, lupa menangani kondisi list kosong, atau tidak menghapus memori node yang sudah tidak dipakai.

Implementasi C++: Struktur Dasar Program

Contoh berikut menggunakan global head dan fungsi-fungsi operasi linked list

```
struct Node {
    int data;
    Node* next;
};

Node* head = NULL;

bool isEmpty() {
    return head == NULL;
}

void insertDepan(int nilai) {
    Node* baru = new Node;
    baru->data = nilai;
    baru->next = head;
    head = baru;
}
```

Poin penting kode

- struct Node menyimpan data dan pointer next
- head adalah penanda node pertama
- isEmpty() memudahkan validasi list kosong
- insertDepan() adalah operasi O(1) karena hanya mengubah head

Prinsip utama: pointer head selalu dijaga agar konsisten setelah insert atau delete.

Kompleksitas Waktu dan Kesalahan Umum

Analisis ini penting agar mahasiswa tidak hanya bisa membuat program, tetapi juga memahami efisiensinya

Kompleksitas operasi

Operasi	Waktu	Catatan
Insert depan	O(1)	hanya ubah head
Insert belakang	O(n)	perlu mencari node terakhir
Traversal/Search	O(n)	cek node satu per satu
Delete depan	O(1)	geser head
Delete belakang	O(n)	cari node sebelum terakhir

Catatan: insert belakang dapat menjadi O(1) jika program juga menyimpan pointer tail.

Kesalahan yang sering terjadi

- lupa set next = NULL pada node terakhir
- tidak menangani kondisi list kosong
- salah memutus atau menyambung pointer next
- menghapus node tetapi tidak memperbarui head
- lupa delete/free sehingga memori bocor

Debug linked list dimulai dari memeriksa panah/pointer.

Ringkasan dan Best Practice

Apa yang perlu diingat saat mengimplementasikan single linked list dalam C++

Best practice implementasi

- Gunakan head sebagai sumber kebenaran posisi awal list.
- Validasi list kosong sebelum traversal, insert tertentu, atau delete.
- Perbarui pointer secara berurutan agar tidak kehilangan referensi node.
- Lepaskan memori node yang dihapus dengan delete.
- Tambahkan tail bila program sering melakukan insert belakang.

Single linked list tepat dipilih ketika struktur data sering berubah dan akses acak tidak menjadi prioritas.

Takeaway utama

Konsep	node + pointer next
Arah traversal	satu arah
Kekuatan	insert/delete fleksibel
Kelemahan	search/traversal O(n)
Implementasi	butuh ketelitian pointer

Tugas Implementasi C++: Lengkapi Program Berikut!

Struktur program praktikum yang memuat insert, search, update, delete, dan clear

```
int main() {
    int pilihan, nilai, key, nilaiBaru;
    do {
        cout << "1.Insert Depan  2.Insert Belakang\n";
        cout << "3.Insert Setelah 4.Tampil 5.Cari\n";
        cout << "6.Update 7.Hapus Depan 8.Hapus Belakang\n";
        cout << "9.Hapus Data 10.Clear 0.Keluar\n";
        cin >> pilihan;

        switch (pilihan) {
            case 1: insertDepan(nilai); break;
            case 2: insertBelakang(nilai); break;
            case 3: insertSetelah(key, nilaiBaru); break;
            case 4: tampil(); break;
            case 5: cariData(key); break;
            case 6: updateData(key, nilaiBaru); break;
            case 7: hapusDepan(); break;
            case 8: hapusBelakang(); break;
            case 9: hapusData(key); break;
            case 10: clearList(); break;
        }
    } while (pilihan != 0);
}
```

Alur interaksi program

Pendekatan menu membantu mahasiswa menguji semua operasi linked list secara bertahap dan melihat dampak perubahan pointer setelah setiap aksi.